Ask What Matters

Adaptive AI for Smarter Travel Reviews

2026 Wharton Hack-AI-thon | Expedia Group

Bailey McIntosh

April 2026

The Core Problem: The Reviewer Side

- Reviewers produce **generic content** — staff friendliness, cleanliness — regardless of what prospective guests actually need
- Surveys and follow-up forms see low compliance; reviewers resist structured prompts that feel like extra work
- Post-stay motivation is low: the reviewer has no stake in downstream traveler decisions
- A 2023 Phocuswright study found $> 60\%$ of OTA reviews fail to mention at least one category the booking traveler later cited as decision-relevant

Core tension: The people who *write* reviews optimize for expressing their experience. The people who *read* reviews optimize for resolving specific uncertainties. These objectives rarely align by accident.

The Core Problem: The Prospective Guest Side

- Guests must infer property attributes from a corpus of *incidentally* written reviews — effectively a **missing-data inference problem**
- High-variance topics (noise levels, breakfast quality) are especially underserved: few reviewers volunteer them, yet they drive booking decisions
- Review volume inflates confidence without improving **topical coverage**: $n = 200$ reviews that all mention location add no information about the pool
- Existing systems (Expedia, TripAdvisor, Booking.com) surface category averages but do not *direct* reviewer attention toward information gaps

Key insight: More reviews do not solve the problem. *Better-directed* reviews do. Can we guide reviewers toward the topics that matter most to future guests, without adding friction or perceived burden?

Thesis: Demand-Driven Reviewer Nudging

If we surface, at the moment of review composition, the precise topics that prospective guests have signaled uncertainty about, reviewers will include that information — not because they are compelled, but because they are shown its value.

Two signals combined:

- **Supply signal** — gap analysis over the existing review corpus: which topics are under-documented or contested?
- **Demand signal** — prospective guest questions (SNSAS): which topics are travelers actively asking about?

Behavioral framing: Reviewers resist being *told* what to write. Framing the nudge as "here is what others are still unsure about" converts review-writing into an act of social contribution, triggering prosocial motivation rather than compliance friction.

Why Existing Approaches Fall Short

Where current approaches fail

- Category rating forms force discrete input and too many controls
- Review template prompts add perceived burden and reduce flow
- NLP post-hoc extraction only analyzes what was already written
- Static FAQ / Q&A content stays isolated from the review corpus
- Sentiment summaries compress tone, not information coverage

How our system departs

- We surface topics without forcing structured answers
- The nudge is non-blocking and visible at a glance
- We intervene *before* composition, not after submission
- Demand signals reshape future review content at the source
- We distinguish missing information from already-covered topics

Key departure: Every existing approach is reactive — it acts on reviews after they are written. Our system acts *during* composition, when the reviewer can still be guided.

System Architecture Overview

Core pipeline

1. Load review corpus, property descriptions, and traveler demand data
2. Score coverage gaps with a 4-signal analyzer
3. Generate a compact "Travelers still want to know" panel
4. Track topic coverage live while the user writes

Feedback loop

- Browse page collects traveler questions (SNSAS)
- Question classifier maps them into topic buckets
- Demand scores feed back into future gap

System components

- **Data:** reviews, property descriptions, traveler_questions.csv
- **Scoring:** embeddings + VADER + LLM + traveler demand
- **Generation:** `gpt-4o-mini` for nudge intro text
- **Interface:** Streamlit — review, browse, and thank-you states

Stack

Streamlit for hosting, OpenAI for generation and embeddings, VADER for fast local sentiment. All expensive steps are cached with `@st.cache_data`.

Data Model: Review Corpus & Time Weighting

Reviews_PROC.csv

- 13 properties, ∼1,600 total reviews
- Key fields: `eg_property_id`, `review_text`, `acquisition_date`, `rating` (JSON dict)
- At load time: `rating` parsed to structured dict; `acquisition_date` coerced to `datetime64`
- **Time-decay weighting** per property:

$$w_i = \exp\left(-\frac{\ln 2}{\tau} \cdot d_i\right)$$

  where d_i = days since review, $\tau = 180$ days (half-life)
- Top-N weighted reviews ($N = 40$) selected for embedding

Why time-decay weighting?

Older reviews describe a hotel that may no longer exist: new ownership, renovations, staff turnover. Exponential decay with a 180-day half-life ensures recent guest experiences dominate gap detection.

Top-$N = 40$ selection: Full embedding of all 1,600 reviews is expensive. Selecting the top-40 by weight is 4× cheaper while retaining the most informative recent signal.

Data Model: Topic Taxonomy & Traveler Questions

Topic Taxonomy — 15 fixed categories

- **10 universal:** Location, Room Quality, Service, Atmosphere, Bathroom, Sleep Quality, Noise, Value, Cleanliness, Check-in
- **5 conditional:** Food, Breakfast, Pool/Facilities, WiFi, Parking — gated on property amenity data
- Each topic has a label, keyword list, icon, and a **dense anchor phrase** for embedding comparison
- Anchor phrases beat short labels: *"hotel noise levels quiet loud soundproofing street traffic walls rooms disturb sleep"* embeds more specifically than *"Noise Levels"*

traveler_questions.csv

Fields: `eg_property_id`, `question`, `topic`, `timestamp`, `source`

Intentionally **never cached** — always read fresh so a question submitted in one browser tab is immediately visible to a reviewer in another tab.

Why anchor phrases? Short labels embed near many irrelevant concepts. Rich descriptive phrases occupy a more specific region of the latent space, improving cosine similarity precision.

TWTK Algorithm: Four-Signal Overview

`analyze_property(property_id, demand=())` returns up to 3 gap topics per property. Four independent signals feed a single scorer.

The four signals

1. **Embedding coverage** — what fraction of recent reviews substantively discuss each topic?
2. **VADER sentiment variance** — are guest opinions on a topic split (contested)?
3. **LLM unique topics** — what property-specific gaps does GPT-4o-mini find that the taxonomy misses?
4. **SNSAS demand** — which topics are prospective guests actively asking about?

Gap types produced

- **gap** — coverage below threshold (< 0.15)
- **contested** — high sentiment variance ($\sigma_{\text{VADER}} \geq 0.38$)
- **demand** — well-covered but travelers are asking
- **unique** — LLM-discovered, outside taxonomy

Filtering

A topic surfaces only if: cov < 0.15 **or** $\sigma \geq 0.38$ **or** $d > 0$

Signal 1: Semantic Coverage via Embeddings

Algorithm

1. Embed top-N weighted reviews with `text-embedding-3-small` (batch call). Result: $R \in \mathbb{R}^{N \times 1536}$, row-normalised.
2. Embed one **anchor phrase** per topic (cached globally). Result: $\mathbf{a}_l \in \mathbb{R}^{1536}$ per label l.
3. Cosine similarity: $\text{sim}_{il} = \hat{r}_i \cdot \hat{a}_l$. Review is *on-topic* iff $\text{sim}_{il} \geq \theta_r = 0.40$.
4. **Coverage** = weighted fraction of on-topic reviews:

$$\text{cov}_l = \frac{\sum_{i:\, \text{sim}_{il} \geq \theta_r} w_i}{\sum_i w_i}$$

5. Flag as *gap* if $\text{cov}_l < \theta_g = 0.15$.

Design decisions

Why review-level here, sentence-level in real-time? Per-sentence embedding gives live keypress feedback. Whole-review embedding in corpus analysis is sufficient and 10× cheaper.

Threshold $\theta_r = 0.40$: Below this, reviews mention a word adjacent to the topic but do not discuss it substantively. Tuned empirically on the 13-property corpus.

Why anchor phrases, not topic labels? Short labels embed near many irrelevant concepts. Rich phrases occupy a more specific latent region.

Signal 2: VADER Sentiment Consensus

Algorithm

1. Collect on-topic texts $\mathcal{T}_l$; require $|\mathcal{T}_l| \geq 4$.
2. Compute VADER compound score $s_t \in [-1, 1]$ for each $t \in \mathcal{T}_l$.
3. Std dev of scores: $\sigma_l = \text{std}(\{s_t\})$. Flag as **contested** if $\sigma_l \geq \theta_c = 0.38$.

```
sentiments = [
    vader.polarity_scores(t)["compound"]
    for t in on_topic_texts
]
if np.std(sentiments) >= CONTESTED_STD:
    is_contested = True
```

Design rationale

What "contested" signals: High variance means some guests rated the topic highly, others poorly. Their concrete experience helps future guests resolve genuine uncertainty.

Why VADER over a transformer? VADER is rule-based and runs in $<$ 1ms per text. At 40 reviews $\times$ 15 topics, a neural model would add $\sim$3–5 seconds per page load.

$\theta_c = 0.38$ **rationale:** VADER compound scores cluster tightly for homogeneous corpora ($\sigma \approx 0.1$–0.2). A threshold of 0.38 fires only on genuine bimodality.

Signal 3: LLM-Discovered Unique Topics

`discover_unique_topics(property_id)`

Motivation: The 15-topic taxonomy covers hotel universals. Each property has idiosyncratic gaps — a safari lodge's *"Big Five sighting probability"*, a mountain hotel's *"Snow conditions"* — that no fixed taxonomy can anticipate.

Prompt inputs: property name, city, description (600 chars), amenities (300 chars), 12 most recent review texts.

Task: Ask GPT-4o-mini what a prospective guest would be *genuinely uncertain* about that the reviews fail to answer. Exclude generic topics; return `[]` if no genuine gaps exist.

Output: JSON array, up to 2 items: `[{"label": "...", "icon": "emoji"}]`

Integration & tradeoffs

Unique topics are appended with a fixed score of 0.2 and are **not** boosted by demand (SNSAS maps to the fixed 15-label taxonomy; unique topics are outside the anchor dictionary).

One GPT-4o-mini call per property, cached. Temperature = 0.3 for reproducible but not mechanical output. Cap at 2 items — beyond 2, the LLM tends to hallucinate relevance.

Guard rails:

- "Do NOT suggest generic topics"
- "Return [] if no genuine gaps exist"
- JSON-only output requirement

Signal 4 & Scoring: Demand Weighting

Demand signal derivation

Normalise SNSAS question counts to $[0, 1]$:

$$d_l = \frac{c_l}{\max_j c_j}$$

where c_l = count of user questions mapped to topic l. The most-asked topic always scores $d_l = 1.0$.

Cache-busting: demand is passed as a sorted hashable tuple, forcing re-computation of `analyze_property` whenever the question set changes:

```
demand_tuple = tuple(sorted(
    get_demand_scores(pid, questions_df).
        items()
))
gaps = analyze_property(pid, demand=
     demand_tuple)
```

Final scoring formula

$$\text{score}_l = \text{base}_l \times (1 + \beta_d \cdot d_l)$$

Base scores by gap type:

$$\text{base}_l = \begin{cases} 1 - \text{cov}_l & \text{gap } (\text{cov} < 0.15) \\ 0.5(1 - \text{cov}_l) & \text{contested} \\ 0.4\, d_l & \text{demand-only} \end{cases}$$

$\beta_d = 3.0$: at this value, a single SNSAS question roughly doubles the score of a mid-strength gap topic.

Top-3 by score are returned. LLM-unique topics are appended at a fixed score of 0.2, not competing with formula-scored topics.

Real-Time Reviewer Feedback: `covered_topics()`

Algorithm

1. Split the in-progress review on `[.!?]+` into sentences of length > 8 chars.
2. For each sentence, call `embed_sentence()` — cached by content, so unchanged sentences incur no API cost.
3. Compute cosine similarity against each gap topic's anchor.
4. A topic is *covered* if any sentence scores $\geq \theta_r = 0.40$.
5. Update the TWTK panel live: covered topics show a green checkmark; footer switches to *"You've covered everything"* when all gaps are addressed.

Latency management

Why sentence-level at keypress? Whole-review re-embedding on each keypress would lag. Per-sentence caching means only new or modified sentences trigger API calls.

Post-submission insights: After posting, `extract_insights()` calls GPT-4o-mini to produce one factual sentence per covered topic, shown on the thank-you page to reinforce the reviewer's contribution.

Example: "Bathroom: Shower has strong water pressure and reliable hot water throughout the day."

Feature 2: Browse Page & SNSAS

Browse page

- Top-10 most recent reviews per property, sorted descending
- Long reviews ($>$ 300 chars) collapsed with *Read more* $\downarrow$ toggle
- Expand/collapse state keyed per review index, persisted in Streamlit session state

SNSAS form (Still Not Sure About Something)

- Free-text question input
- On submit: classify $\rightarrow$ append to CSV $\rightarrow$ clear text area (via counter-keyed widget key)
- Wipe button resets demo data (`source="user"` rows only)

`classify_question_topic(question)`

Maps free-text questions to one of 15 canonical labels:

1. Embed question: $\mathbf{q} \in \mathbb{R}^{1536}$
2. Find highest-similarity anchor: $\hat{l} = \arg\max_l \ \hat{\mathbf{q}} \cdot \hat{\mathbf{a}}_l$
3. Return that label

No threshold cutoff — every question maps somewhere. Edge cases degrade gracefully to the nearest category.

Cached by question string, so repeated demo submissions are free.

The Demand Feedback Loop: End to End

1. A prospective guest browses a property page and submits a question
2. `classify_question_topic()` maps it to a topic label
3. The question is appended to `traveler_questions.csv`
4. `get_demand_scores()` converts topic counts to normalised demand d_l
5. The next reviewer's TWTK panel is re-ranked using that demand signal
6. New reviews then address the topics travelers are actively asking about

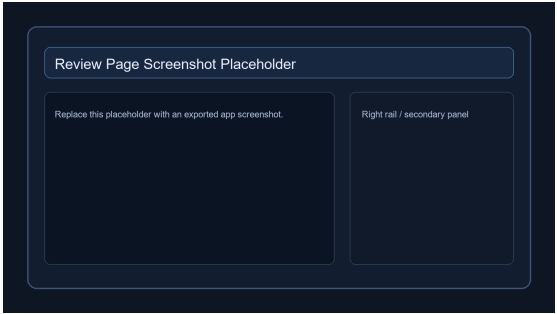
Temporal dynamics

Questions accumulate over time. The demand signal strengthens as more guests ask the same thing, creating urgency proportional to collective uncertainty rather than individual preference.

Live during the demo

`load_questions()` is intentionally uncached, so a question submitted in one browser tab is immediately visible to a reviewer in another tab.

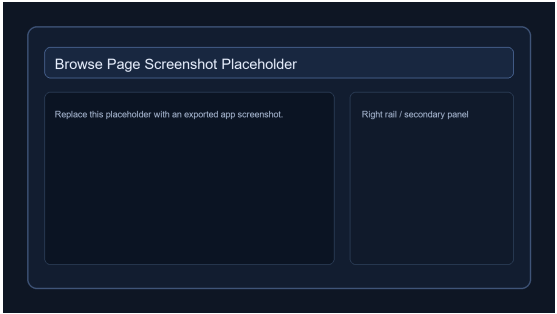
Product Demo: Feature 1 — Review Page



Annotated elements

1. **Property banner** — name, location, Expedia rating
2. **Review text area** — monitored live for topic coverage
3. **TWTK panel** (right column):
   - GPT-4o-mini intro sentence per property
   - Per-topic badges: Travelers asking / Mixed reviews / Not documented
4. **"Why am I seeing this?"** expander — transparency
5. **Post review** primary action button

Product Demo: Feature 2 — Browse Page



Annotated elements

1. **Review cards** — dark surface cards with *Read more* ↓ toggle for long reviews
2. **Rating badge** — overall score / 5
3. **SNSAS form:**
   - Free-text input
   - Submit: embed → classify → save to CSV
   - Text area clears via counter-keyed widget
4. Question count + Wipe button for demo resets
5. Data written here feeds directly into the demand multiplier on the reviewer side

Technical Summary

Stack

- Frontend / hosting: Streamlit (Python)
- Embeddings: `text-embedding-3-small`
- Language model: `gpt-4o-mini`
- Sentiment: VADER (local, <1ms/text)
- Data: Pandas / NumPy / CSV
- Caching: `@st.cache_data` / `cache_resource`

Key parameters

- $\theta_r = 0.40$ — topic relevance threshold
- $\theta_g = 0.15$ — coverage gap threshold
- $\theta_c = 0.38$ — contested sentiment std dev
- $\tau = 180$ days — decay half-life

Information flows

1. Corpus → embeddings → coverage scores
2. Corpus → VADER → contested flags
3. Corpus + property desc → LLM → unique topics
4. SNSAS questions → classifier → demand scores
5. All signals → scorer → TWTK top-3
6. Reviewer text → per-sentence embeddings → live feedback
7. Submitted review → LLM → per-topic insights on thank-you page